

为什么2025年是通用Agent元年？

Agent技术架构全梳理

赵健 PPIO 布道师
2025.08.21

1.Agent核心技术栈

2.如何构建Agent?

3.记忆与上下文工程

4.PPIO Agent Infra

彩蛋: Agent最全面的资料干货

为什么关注Agent?

Sam Altman: “未来工作的基本单位，是AI Agent。”



Agent最大的特点是“主观能动性”

What is an agent?

While conventional software enables users to streamline and automate workflows, agents are able to perform the same workflows on the users' behalf with a high degree of independence.

Agents are systems that **independently** accomplish tasks on your behalf.

OpenAI

What is an agent?

In its most fundamental form, a Generative AI agent can be defined as an application that attempts to achieve a goal by observing the world and acting upon it using the tools that it has at its disposal. Agents are autonomous and can act independently of human intervention, especially when provided with proper goals or objectives they are meant to achieve. Agents can also be proactive in their approach to reaching their goals. Even in the absence of explicit instruction sets from a human, an agent can reason about what it should do next to achieve its ultimate goal. While the notion of agents in AI is quite general and powerful, this whitepaper focuses on the specific types of agents that Generative AI models are capable of building at the time of publication.

Google

What are agents?

"Agent" can be defined in several ways. Some customers define agents as fully autonomous systems that operate independently over extended periods, using various tools to accomplish complex tasks. Others use the term to describe more prescriptive implementations that follow predefined workflows. At Anthropic, we categorize all these variations as **agentic systems**, but draw an important architectural distinction between **workflows** and **agents**:

- **Workflows** are systems where LLMs and tools are orchestrated through predefined code paths.
- **Agents**, on the other hand, are systems where LLMs dynamically direct their own processes and tool usage, maintaining control over how they accomplish tasks.

Below, we will explore both types of agentic systems in detail. In Appendix 1 ("Agents in Practice"), we describe two domains where customers have found particular value in using these kinds of systems.

Anthropic

What are agents? 🤖

An introduction to agentic systems.

Any efficient system using AI will need to provide LLMs some kind of access to the real world: for instance the possibility to call a search tool to get external information, or to act on certain programs in order to solve a task. In other words, LLMs should have **agency**. Agentic programs are the gateway to the outside world for LLMs.

AI Agents are programs where LLM outputs control the workflow.

Any system leveraging LLMs will integrate the LLM outputs into code. The influence of the LLM's input on the code workflow is the level of agency of LLMs in the system.

Hugging Face

Workflow vs Agent

1. Automated workflow

- 1.自动邮件回复系统
- 2.早期的客服机器人
- 3.智能音箱

2. AI workflow

- 1.Chatbot

3. Agentic Workflow:

- 1.DeepResearch
- 2.Manus

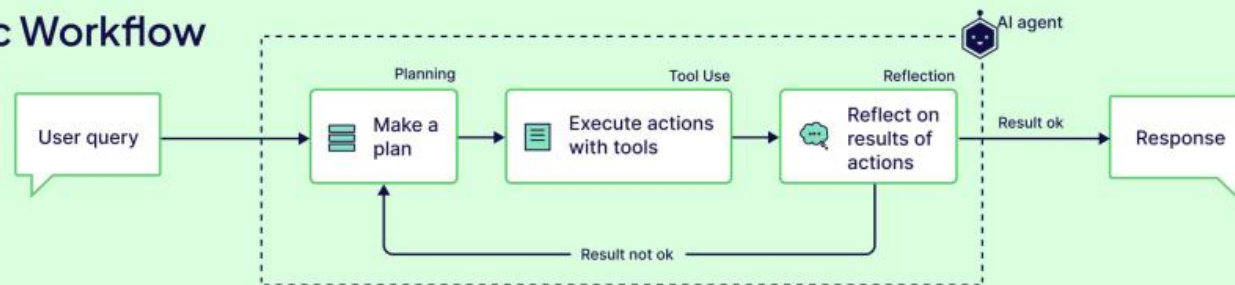
Automated Workflow (rule-based, non-AI)



AI Workflow (non-agentic)



Agentic Workflow



Agent分级

OpenAI:

- L1: Chatbots, 对话能力
- L2: Reasoners, 人类水平的问题解决能力
- L3: Agents, 能够采取行动的系統
- L4: 创新者, 能够辅助发明的人工智能
- L5: 组织, 能够完成组织工作的人工智能

Hugging Face:

- 简单处理器: LLM输出对程序流程没有影响
- 路由器: LLM输出控制if/else开关
- 工具调用: LLM输出控制函数执行
- 多步代理: LLM输出控制迭代和程序继续
- 多代理: 一个代理工作流可以启动另一个代理工作流
- 代码代理: LLM在代码中起作用, 可以定义自己的工具/启动其他代理

OpenAI Imagines Our AI Future

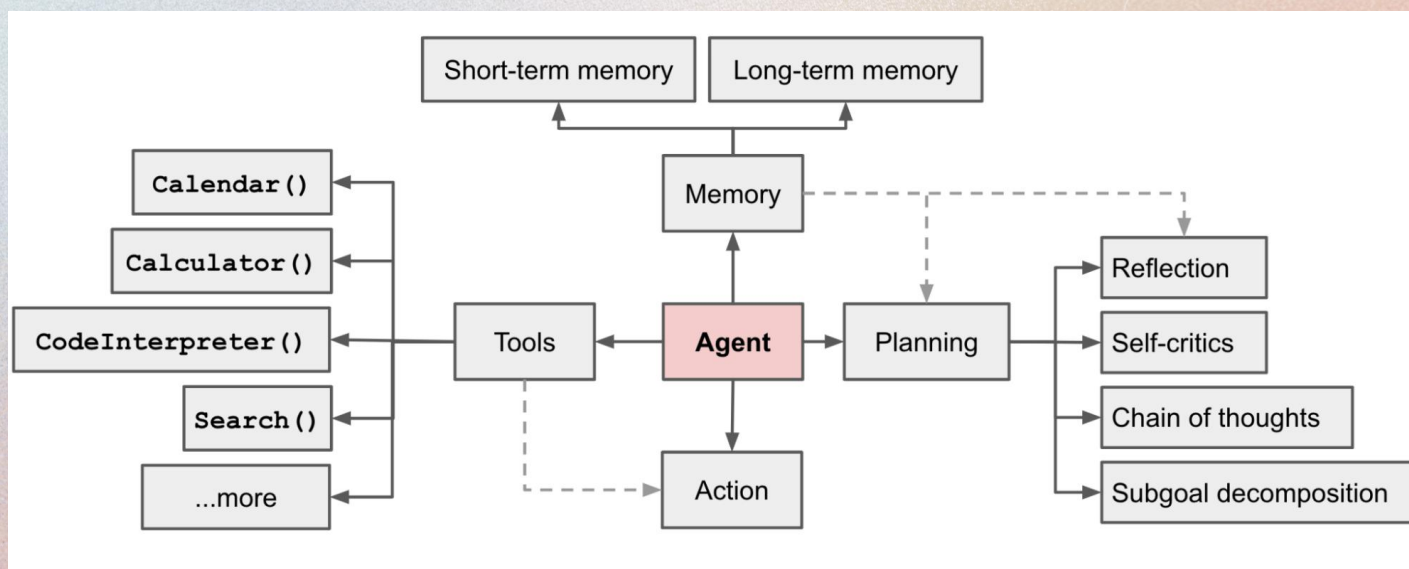
Stages of Artificial Intelligence

Level 1	Chatbots, AI with conversational language
Level 2	Reasoners, human-level problem solving
Level 3	Agents, systems that can take actions
Level 4	Innovators, AI that can aid in invention
Level 5	Organizations, AI that can do the work of an organization

Source: Bloomberg reporting

Agency Level	Description	Short name	Example Code
☆☆☆	LLM output has no impact on program flow	Simple processor	<code>process_llm_output(llm_response)</code>
★☆☆	LLM output controls an if/else switch	Router	<code>if llm_decision(): path_a() else: path_b()</code>
★★☆	LLM output controls function execution	Tool call	<code>run_function(llm_chosen_tool, llm_chosen_args)</code>
★★☆	LLM output controls iteration and program continuation	Multi-step Agent	<code>while llm_should_continue(): execute_next_step()</code>
★★★	One agentic workflow can start another agentic workflow	Multi-Agent	<code>if llm_trigger(): execute_agent()</code>
★★★	LLM acts in code, can define its own tools / start other agents	Code Agents	<code>def custom_tool(args): ...</code>

Agent基本架构



2023年，OpenAI研究员翁荔定义了Agent的四大基础组件：
Planning、Memory、Tools、Action

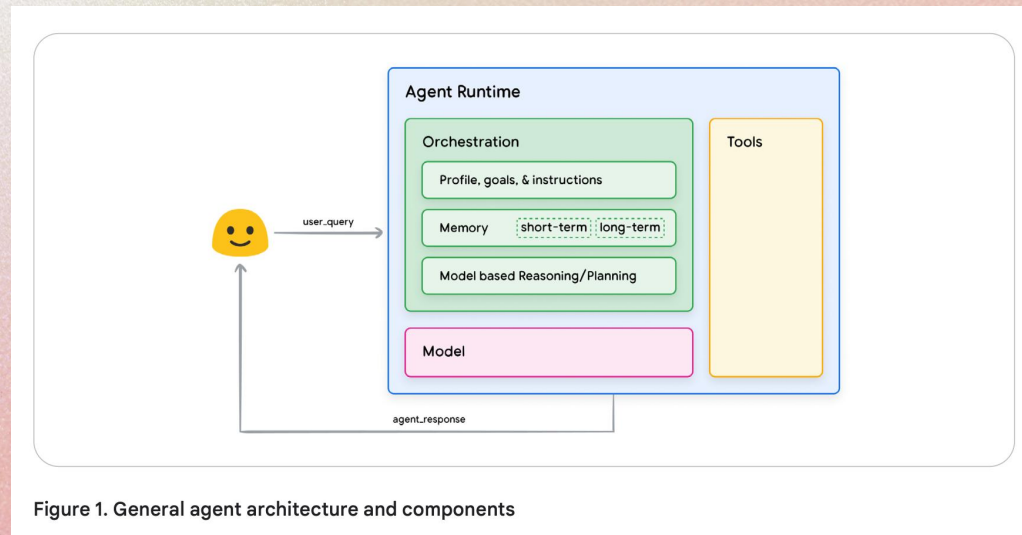
Agent核心组件

Agent design foundations

In its most fundamental form, an agent consists of three core components:

01	Model	The LLM powering the agent's reasoning and decision-making
02	Tools	External functions or APIs the agent can use to take action
03	Instructions	Explicit guidelines and guardrails defining how the agent behaves

OpenAI Agent框架: Model+Tools+Instrctions

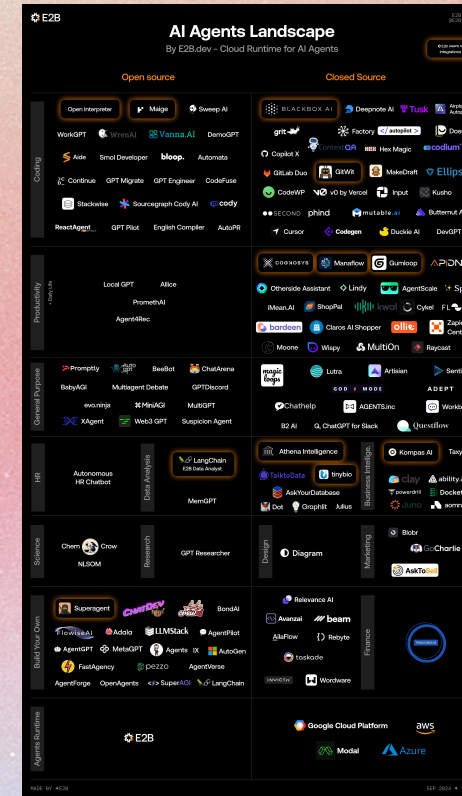
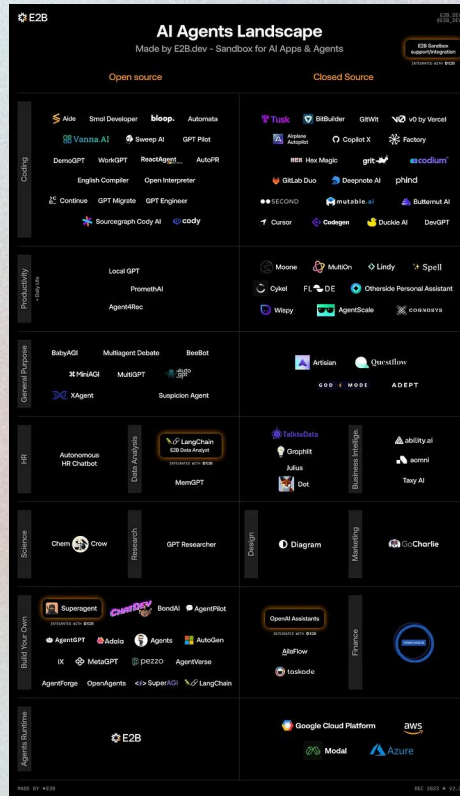


Google定义的Agent框架:
Model+Tools+Orchestration

Agent Models

2023: GPT-4

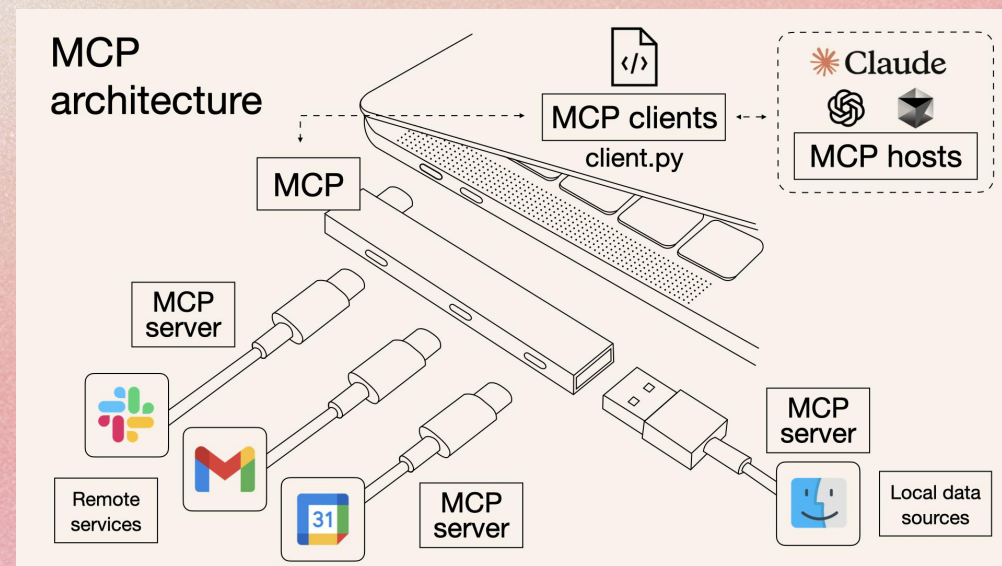
2024: o1、Claude 3.5



Tool Use

Function Call (调用插件)
Web search (网络搜索)
File search (本地搜索)
Computer use (操作电脑)
Browser Use (操作网页)

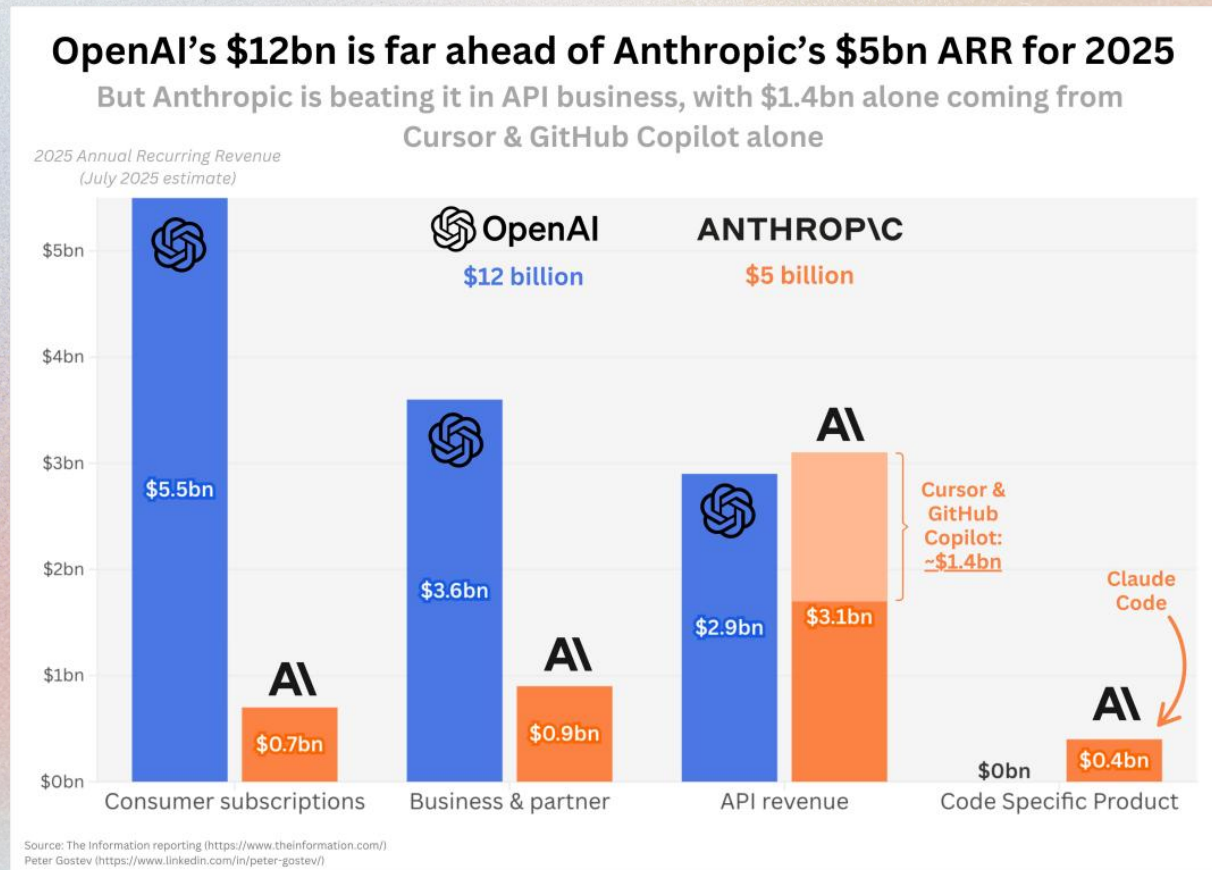
模型自带的基础工具



MCP协议支持海量的扩展工具

OpenAI: Reasoning Models

Anthropic: Coding Models



Agent产品框架/工具

1. Monitoring, Observability, Analytics (监控、可观测性与分析)
2. Frontend (前端工具)
3. Runtime for LLMs (LLM 运行时环境)
4. Building Frameworks & Platforms (构建框架与平台)
5. Data Integration, Memory Management (数据集成与记忆管理)
6. API and Routers for LLMs (LLM API 与路由器)
7. Libraries for Building AI Products (AI 应用开发库)
8. Orchestration (编排)



Monitoring

Agent Runtime Environments

Orchestration

Profile, goals & instructions

Memory

short-
term

long-term

Reasoning/Planning

API Routers

Models

Tools

Function Call
Web search
File search
Computer use
Browser Use

MCP

Deep Research 是第一个通用Agent

AutoGPT / BabyAGI

LangChain / LlamaIndex

Anthropic Claude 3.5、OpenAI o1

谷歌 Gemini 1.5 + NotebookLM

Deep Research

Why Deep Research first?

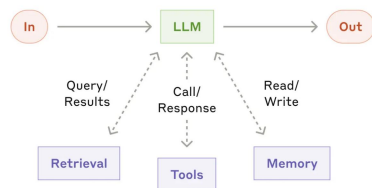
需求端:

1. 信息收集是知识工作者的普遍需求。
2. 人们在日常工作中有处理不同的文件格式的需求。
3. 人们经常多个应用程序之间切换。

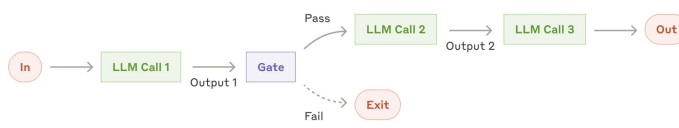
技术端:

1. 稳定闭环: 引入 Orchestration (规划) + 工具链调度 + 上下文管理, 能稳定跑几十甚至上百步。
2. 记忆与搜索结合: 结合长上下文和 RAG, 让推理链保持一致性。
3. 工具生态: 不仅 function call, 还支持文件检索、web search、API routing 等, 形成相对完整的 action space。

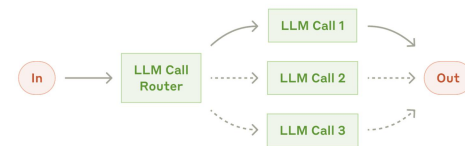
如何构建一个Agent?



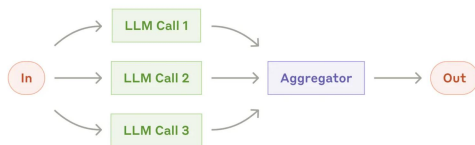
Building block: The augmented LLM
增强型LLM



Workflow: Prompt chaining
链式提示



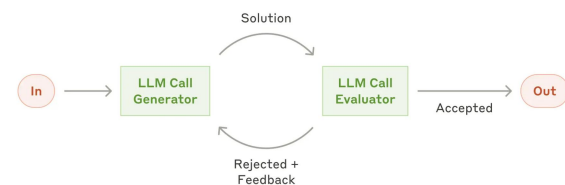
Workflow: Routing
路由



Workflow: Parallelization
并行化

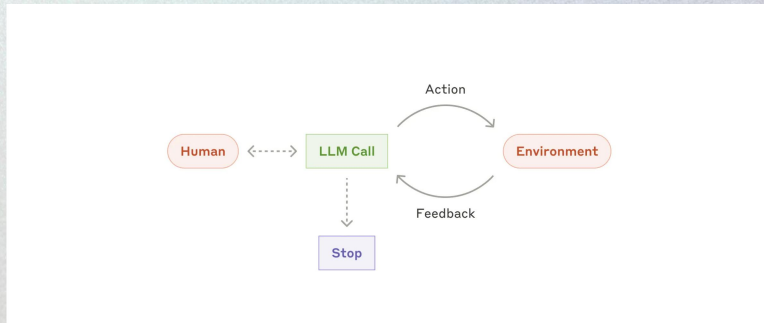


Workflow: Orchestrator-workers
协调器编排

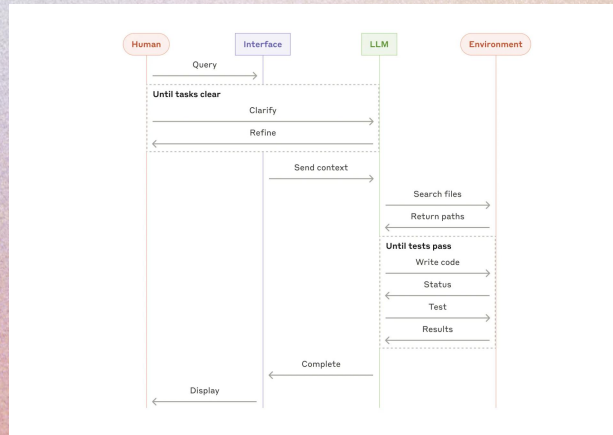


Workflow: Evaluator-optimizer
评估器-优化器

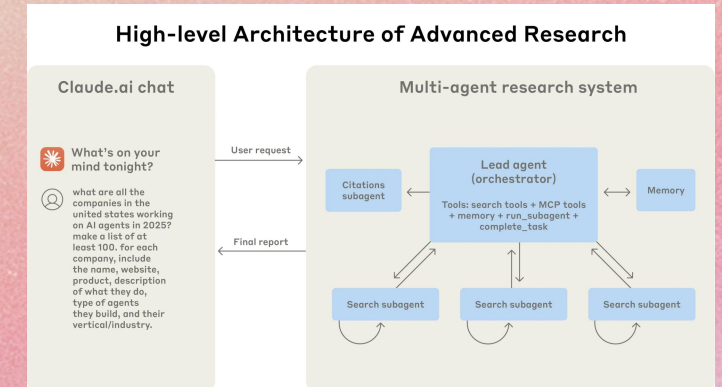
如何构建一个Agent?



Autonomous agent



High-level flow of a coding agent



The multi-agent architecture in action

记忆和上下文工程将是下一阶段重点

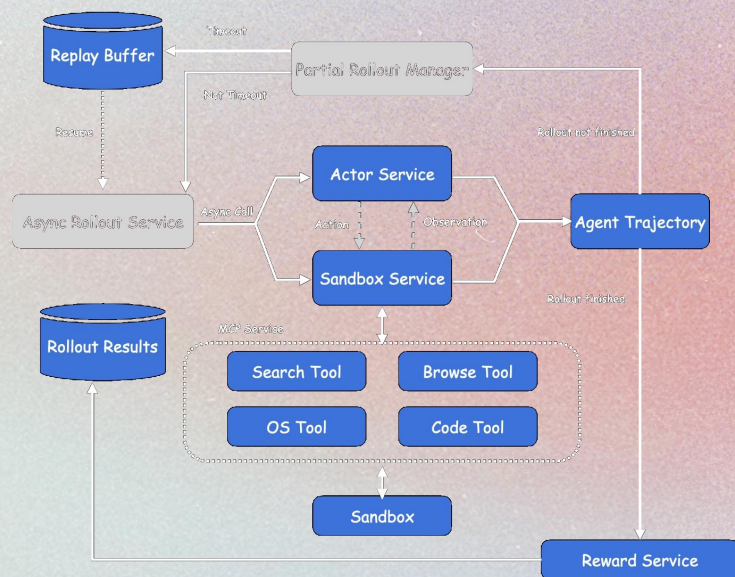
Model Agent

模型厂商代表产品: OpenAI Deep Research,
Kimi Research

训练方法: 端到端强化学习

理念: Model as Agent

Tool use: 训练到模型中



独立Agent

独立Agent代表产品: Manus、Genspark

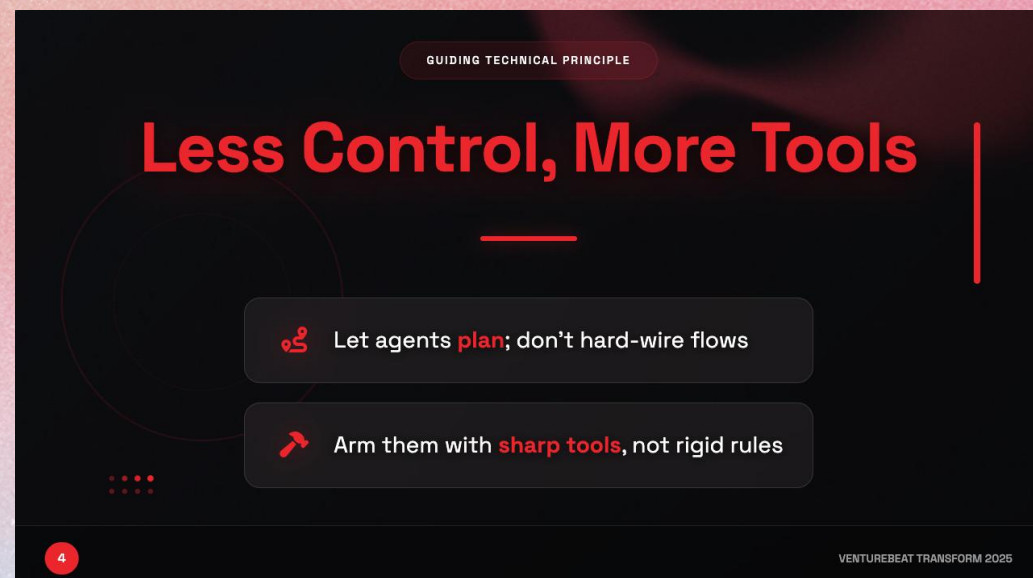
训练方法: Context Engineering (上下文工程)

理念:

Manus: less structure more intelligence

Genspark: Less Control, More Tools

Tool use: MCP协议调用



Manus的上下文工程实践

围绕KV缓存进行设计

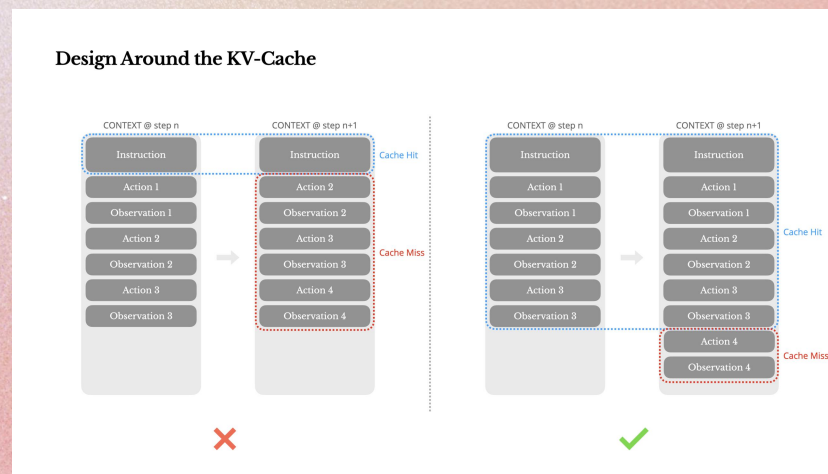
工具使用：遮蔽，而非移除

使用文件系统作为上下文

通过复述操控注意力

保留错误的内容

不要被少样本示例所困



Agent的9种记忆系统

- 顺序记忆
- 滑动窗口
- 基于摘要的优化
- 压缩和整合记忆
- 基于检索的内存 (RAG)
- 内存增强 Transformers
- 多任务的分层优化
- 基于知识图谱的优化
- 类似操作系统的记忆管理

technique name	level	advantage	drawback
Sequential	Beginner	Simple implementation	Very high cost
Sliding Window	Beginner	Cost-effective	Forgets old context
Summarization	Intermediate	Reduces token usage	Loses factual details
Retrieval Based	Advanced	Highly relevant context	Complex implementation
Memory Augmented Transformers	Advanced	Retains critical facts	Expensive extra calls
Hierarchical	Advanced	Hybrid and sophisticated	Complex to manage
Graph Based	Advanced	Understands relationships	Difficult to populate
Compression & Consolidation	Intermediate	Extremely token-efficient	Loses nuance
OS-Like Memory Management	Advanced	Limitless memory potential	Conceptually complex

PPIO在做什么？ ——Agent Infra平台

Model API

GPU云

Sandbox

通用 Agent 产品对运行环境提出了新要求

通用 Agent 特点	对运行环境的要求
自主决策能力提升 任务不可控，执行的代码可能不可信问题	安全隔离、杜绝越权访问
多 Agent 协同框架，工具链动态调用 需要编写或运行代码/ 使用浏览器/ 调用本地工具等	完整的运行环境
用户任务异步性 区别于 chatbot 的同步性，通用 Agent 支持异步任务	跨会话记忆、任务上下文快速恢复
高效处理用户通用任务（复杂、开放性、长期运行） 为每个用户 session 启动一台“电脑”	高并发、弹性伸缩的能力
让用户“看见”实现过程，展示执行过程 如Deepseek，“思考”过程的可见行提高信任度	提供实时的远程桌面流

这对这台“电脑”提出了新的要求：既要安全可控，又要有完整的运行环境，同时支持高并发，具备弹性伸缩能力。

为什么“专属电脑”是 Agent 沙箱？

为什么是「Agent 沙箱」，而不是传统容器或虚拟机？

Agent 沙箱是一个轻量化虚拟机，相比容器，隔离性更好，相比虚拟机，启动速度更快

	Agent 沙箱	容器	虚拟机
- 启动速度 高并发及实时响应	<200ms (MicroVM冷启动)	2- 5秒 (共享内核调度延迟)	30秒+ (完整OS引导)
- 安全隔离 不可预测代码执行	强隔离	弱隔离 (内核共享, CVE漏洞可穿透)	强隔离
- 状态持久性 用户任务异步性	暂停/恢复功能 快照秒级保存	无状态保持 (停止即销毁)	分钟级快照 (存储开销大, 恢复慢)
- 开发者体验	Agent 友好的API接口, 易集成	容器接口, 不是为Agent 设计	VM 接口, 仅资源交付

Agent 沙箱= 启动快/高弹性 + 强隔离 + 状态持久+ Agent 友好 API, 是专为 Agent 设计的新运行基座

PPIO Agent 沙箱：运行在云端的 Agent 沙箱

PPIO Agent 沙箱是专为 AI Agent 执行任务设计的云端运行环境，基于 Firecracker MicroVM 构建，具备ms级启动、vm级隔离等特性，无需预部署，即启即用。可用于 AI Agent 代码执行、信息搜集及其他需要系统权限的任务。

完全兼容 E2B 协议接口，适配主流 Agent 架构。

三大核心优势：更快、更弹性、低成本

更快

- 通过模版提升部署效率
- 通过 ms 级启动速度，提升业务效率

复用率90%

零冷启动

更弹性

- vCPU及内存规格可自定义
- 支持高并发、按需弹性调度

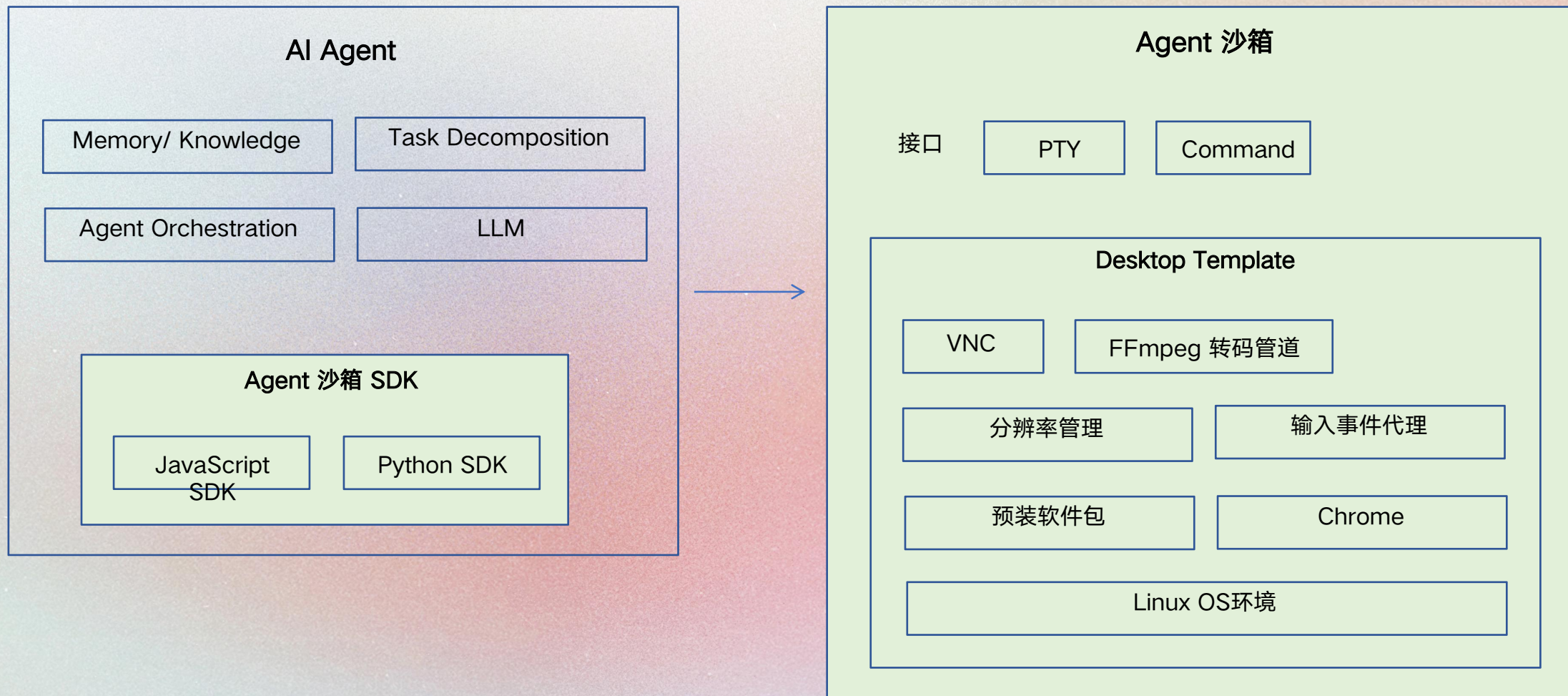
动态成本优化

低成本

- 按照 vCPU 和内存秒级计费
- 支持暂停/恢复沙箱，暂停期间不计费

计算成本节省40%

PPIO Agent 沙箱架构



Template 可根据业务情况自定义

PPIO Agent 沙箱能力

能力类型	说明
代码执行	支持多语言（如 Python、JavaScript、C++）脚本运行
预装多种工具	支持代码解释器、终端、浏览器、office 套件、远程挂载工具
可视输出	支持 vnc 流式输出和实时界面操作，便于Agent 给用户展示任务执行
网络访问	可安全访问外部 API 和公网资源，适配实时数据抓取、联网搜索等任务场景
文件系统	支持上传、下载、读写和监听等操作，使 Agent 能处理较大或异步的本地/远程文件任务
浏览器访问 (Browser Use)	集成浏览器控制接口，支持自动打开网页、填写表单、抓取页面内容，适配复杂页面交互流程
计算机使用 (Computer Use)	可模拟完整计算机操作桌面，包括文本输入、页面滚动、截图等，使 Agent 能完成类人操作任务

PPIO Agent 沙箱 vs E2B Sandbox

PPIO Agent 沙箱：更低门槛，更高稳定性，无需订阅，即可享受 E2B 付费高级服务

	PPIO Agent 沙箱	E2B Sandbox
计费模式	按需计费	订阅费+按需计费
沙箱并发限制	默认提供 50 个沙箱并发	免费版：20 个沙箱 订阅版：100 个沙箱
沙箱生命周期	默认 24 小时	免费版：1 小时 订阅版：24 小时
沙箱存储	默认 20 GB	免费版：10 GB 付费版：20 GB
大内存、大镜像	支持，且稳定	支持，但容易崩溃

费用示例

示例规格+运行时长	PPIO 费用	E2B 费用
1核1GiB 连续运行5分钟	¥ 0.0135	¥ 0.0389
规格：1C1G 连续运行2小时	¥ 0.324	¥ 0.9336+1050（订阅费）

Hobby

Free

+ USAGE COSTS

✓ One-time \$100 of usage in credits

✓ Community support

✓ Up to 1-hour sandbox session length

✓ Up to 20 concurrently running sandboxes

START FOR FREE

(v) NO CC REQUIRED

Pro

\$150 /MO

+ USAGE COSTS

✓ Everything in the Hobby tier

✓ Customize your [Sandbox CPU & RAM](#)

✓ Up to 24-hour sandbox session length

✓ Up to 100 concurrently running sandboxes

CHOOSE PRO

(S) PAYMENT BY STRIPE

CPU RAM STORAGE		
vCPUs	Plan	Costs
1	Pro	\$0.000014/s
2 [Default]	Hobby / Pro	\$0.000028/s
3	Pro	\$0.000042/s
4	Pro	\$0.000056/s
5	Pro	\$0.000070/s
6	Pro	\$0.000084/s
7	Pro	\$0.000098/s
8	Pro	\$0.000112/s

CPU RAM STORAGE	
Memory	Plan
even value between 512 MiB and 8,192 MiB	Hobby / Pro
Costs (both plans)	
\$0.0000045/GiB/s	

诚邀测试

专属支持服务

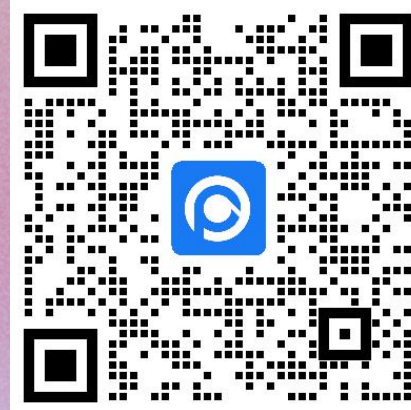
- 测试激励：代金券、资源抵扣额度
- 技术支持：1V多技术对接群，快速响应、深度协作
- 专属服务：产品经理对接、问题优先处理

使用指南

- 帮助文档：<https://ppio.com/docs/sandbox/overview>
- 控制台：<https://ppio.com/sandbox/console>
- 快速入门：<https://ppio.com/docs/sandbox/get-start>

联系我们

- 官网：<https://ppio.com>
- 扫码获取Agent文档下载



Thank you!

赵健 PPIO运营经理